

Instructivo de laboratorio

Laboratorio 3: Microcontrolador parte 2

Profesor

Kaled Alfaro Badilla, M.Sc.

Basado en instructivo con autoría de Dr.-Ing. Pablo Alvarado Moya

Jeferson González Gómez, M.Sc.

Dr.-Ing. Jorge Castro-Godínez

Roberto Molina Robles, M.Sc.

1. Introducción

El laboratorio 3 constituye es una actividad integradora de conceptos en diseño de sistemas digitales y software. El proyecto usa como punto de partida el proyecto anterior. En esta ocasión se incluire un sistema de comunicación serial SPI además de los periféricos ya incluidos. Este protocolo serial SPI será conectado con el acelerómetro ADXL362 encontrado en la NEXYS 4. La idea es utilizar la Nexys 4 como control de una aplicación en la laptop.

La complejidad del sistema a implementar solo es manejable con disciplina de implementación: cada módulo debe tener las pruebas (*testbenches*) necesarias para asegurar su funcionamiento correcto. Esto es relevante, en particular, para el microcontrolador implementado, pues de otra forma no se podrá asegurar que el programa implementado esté ejecutándose de manera correcta en la FPGA.

Para el desarrollo de este laboratorio se deberán acatar las siguientes indicaciones:

1. Lea y trate de comprender todo el trabajo solicitado antes de iniciarlo.
2. Durante el desarrollo del diseño y la validación, asegúrese de documentar todas las tablas de verdad, circuitos, figuras, diagramas (flujo, estado, tiempos), entre otros, que sean necesarios.
3. Para la presentación funcional, se le pedirá que muestre algunos de los circuitos propuestos, ya sea en implementación o simulación. Dicha selección se hará el día de la presentación.

4. Asegúrese de que el avance del proyecto y los aportes de cada estudiante queden debidamente protocolados con suficientes “commit” en el repositorio a lo largo de todo el periodo disponible para trabajar en el laboratorio.
5. Todos los bloques de diseño deben ser **sintetizables** en SystemVerilog. Además, debe seguir la guía de codificación y mejores prácticas.

2. Objetivos

1. Utilizar las herramientas del diseño digital para construir un sistema basado en un microcontrolador, lo que incluye un componente de software programado en lenguaje ensamblador.
2. Afinar la implementación de protocolos de comunicación serie SPI.
3. Investigar y configurar un sensor para su utilización en un sistema.
4. Plantear una estrategia de trabajo en equipo adecuada para diseñar un sistema complejo e implementarlo con restricciones de tiempo existentes.

3. Investigación previa

Para el desarrollo de este proyecto se debe indagar sobre los siguiente aspectos:

1. Investigue el protocolo SPI y cómo configuraría los registros de control para usarlo.
2. Investigue sobre la hoja de datos del sensor ADXL362 y determine los registros de configuración del mismo.
3. Investigue como se relaciona las señales de los ejes del acelerómetro con las posiciones X, Y y Z de los ejes.
4. Investigue que aplicación interactiva puede enlazarse un control que utilice un acelerómetro.

4. Procedimiento

A continuación se describen aspectos fundamentales para el desarrollo de las diferentes etapas de este proyecto.

4.1. Sistema completo

Este laboratorio se construye con la base del laboratorio pasado. La Figura 1 muestra una vista de alto nivel del sistema que se deberá implementar con una FPGA. El sistema debe contar con un método de comunicación UART y SPI. El sistema deberá ser implementado utilizando una FPGA Nexys4 del laboratorio.

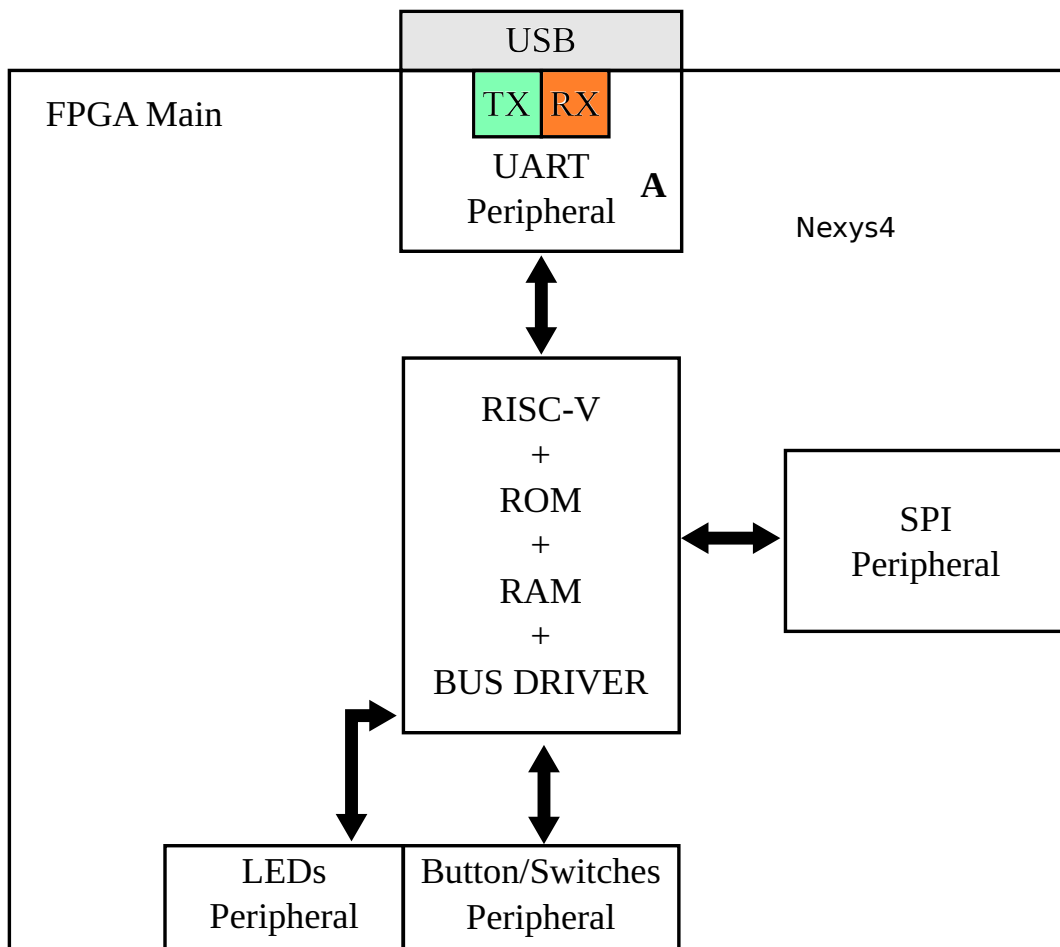


Figura 1: Diagrama de alto nivel del sistema completo (tarjetas de desarrollo).

4.2. Microcontrolador

La Figura 2 muestra los componentes principales del sistema computacional a desarrollar. El microcontrolador utilizará una memoria ROM para almacenar el programa y una memoria RAM para el almacenar los datos. Dichas memorias se podrán acceder mediante buses **independientes**. Como se observa en la Figura 3, estos buses corresponden a ProgAddress_o[31:0] y ProgIn_i[31:0] para la memoria de instrucciones, y DataAddress_o[31:0], DataOut_o[31:0] y DataIn_i[31:0] para la memoria de datos.

Los datos a los que el microcontrolador tiene acceso (read) pueden provenir tanto de la memoria RAM de datos como de los módulos periféricos descritos en este documento, como se representa en la Figura 2.

El microcontrolador deberá operar como mínimo a una frecuencia de 10 *MHz*. Dicha señal de reloj deberá entrar al diseño mediante la entrada *clk_i* (Figura 3). Esta señal de reloj también es común con los otros bloques del sistema. Genérela utilizando un PLL.

La Figura 4 muestra el mapa de memoria considerado para el diseño de este sistema computacional. La memoria de programa tiene un tamaño recomendado de 512 palabras de instrucción, i.e., 2 KiB. Esta memoria de programa debe colocarse en el espacio entre 0x0000 y 0x0fff en el mapa de memoria, siempre iniciando en la posición 0x0000. La memoria de datos posee un tamaño de 100 KiB, y se debe colocar en el espacio de memoria entre 0x40000 y hasta 0x7ffff. Las siguientes direcciones en memoria corresponden a los registros internos de los periféricos:

- Switches/Botones: registro de datos en 0x02000
- LEDs: registro de datos: 0x02004

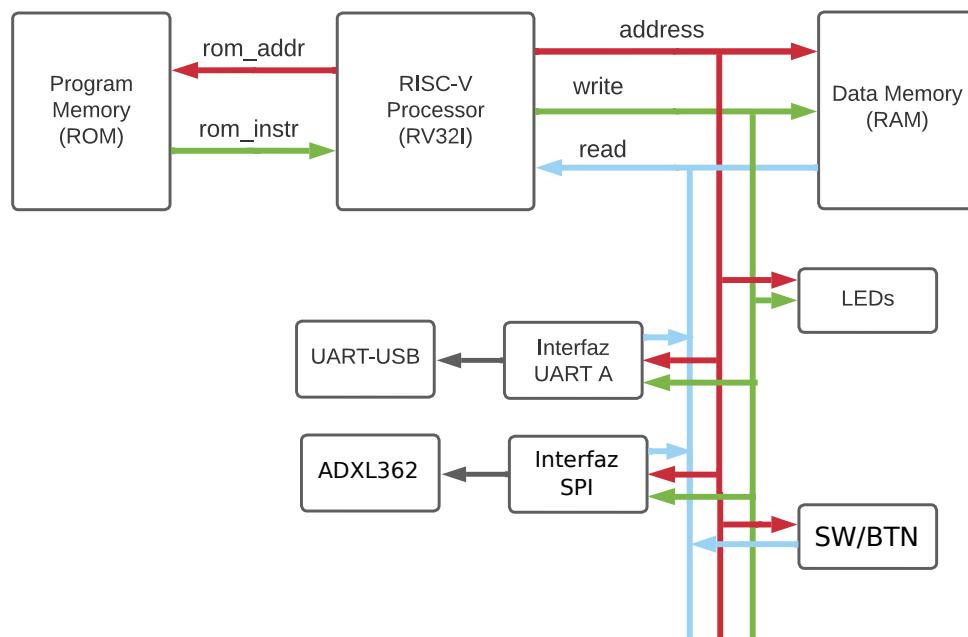


Figura 2: Diagrama de bloques del sistema computacional a desarrollar.

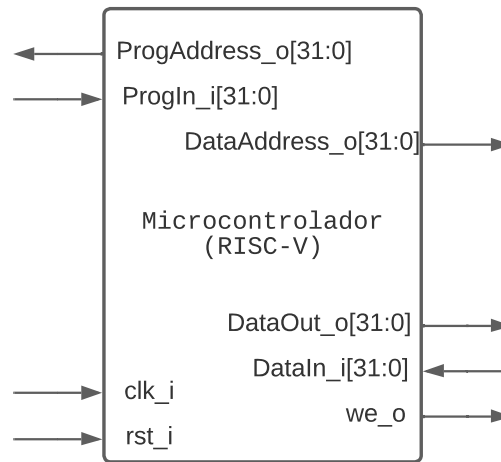


Figura 3: Diagrama de alto nivel del core para el microcontrolador de referencia.

- UART: registro de control en 0x02010, registro de datos en 0x02018 y 0x0201C
- SPI: registro de control en 0x02020, registro de datos en 0x02028 y 0x0202C

4.3. Periféricos

El microcontrolador a desarrollar debe ser capaz de comunicarse con los siguientes periféricos: Interfaz serie RS-232, LEDs y Switches/Botones. Para el desarrollo de estos periféricos y sus interfaces se recomienda hacer referencia al diseño planteado en el instructivo del laboratorio 1, particularmente para la interfaz serial, además de tomar previsiones con respecto a circuitos de acondicionamiento necesarios (e.g., sincronizador y anti-rebotes).

La comunicación entre el procesador, los periféricos y la memoria de datos se muestra de manera simplificada en la Figura 2. Esta se realiza por medio de un bloque que administre los buses de datos (entrada y salida), la señal de write enable y el bus de direcciones, de tal manera que multiplexe las señales de acuerdo a lo establecido en el mapa de memoria (Figura 4).

- **Switches/Botones:** Este bloque permite mapear los 16 botones o switches de la nexys a un registro accesible por el procesador. Este módulo debe considerar **sincronización y anti-rebote**. Este registro debe únicamente actualizarse al recibir un pulso válido del usuario.
- **LEDs:** Este bloque permite mapear los LEDs de la tarjeta a un registro accesible por el procesador. Colóquelos como los 16 bits menos significativos del dato en el registro.
- **SPI-sensor:** Este bloque permite leer y escribir los datos del sensor ADXL362 ().
- **UART:** Este bloque permite mapear una interfaz Serial con el procesador, y cuenta con un registro de control y dos de datos. Uno de los bits de este registro de control permite indicarle a la interfaz que se debe enviar un nuevo dato por medio del protocolo RS-232 dependiendo a cual bloque sea dirigido el registro, mientras que otro bit de dicho registro permite identificar la recepción de un byte.

4.4. Aplicación

4.4.1. Aplicación en la laptop

Se buscará desarrollar o adaptar alguna aplicación de software existente para que utilice de mando de control la nexys4.

4.4.2. Aplicación en el sistema empujado

El funcionamiento de la aplicación de software que corre sobre el sistema principal en la FPGA se describe a continuación:

1. Al inicio el procesador deberá indicar por medio de los LEDs que el procesador RISC-V se encuentra listo para recibir datos.
2. En caso de recibir un paquete desde el puerto serial UART se empezará a realizar a leer la cola de datos recibidos por el UART.
3. Se espera recibir un demarcador de inicio donde indique el inicio de envío de lecturas del acelerómetro.
4. Se estará enviando las tres variables X, Y y Z del acelerómetro cada 10ms por la UART hasta acabar que recibir un demarcador de fin.

4.5. Implementación del programa en ensamblador

Para el desarrollo del programa requerido, se sugiere el uso de un intérprete del ensamblador de RISC-V, particularmente para el subconjunto de instrucciones RV32I. Si bien la implementación que desarrolle con un intérprete requerirá de modificaciones para acceder a los dispositivos externos, le permitirá:

1. Determinar las instrucciones necesarias para su programa y para las cuales el microcontrolador debe brindar el soporte requerido.
2. Validar el funcionamiento de la base de su programa.
3. Generar el código máquina a partir del programa base en lenguaje ensamblador.

El programa debe ser capaz de correr indefinidamente en el microcontrolador desarrollado siguiendo el algoritmo descrito en la Sección 4.4.

5. Evaluación

Las fechas oficiales para este laboratorio son las siguientes. Recuerde que para las revisiones, debe contar con simulaciones post-síntesis con temporización e implementación en FPGA.

- **Inicio:** 07/05/2026

■ **Revisión final:** 11/06/2026

El presente laboratorio tiene un valor del **30 %** de la nota final del curso y se evaluará de la siguiente manera:

- | | |
|---|------|
| 1. Funcionalidad final: Aplicación | 60 % |
| 2. Documentación técnica del sistema desarrollado | 20 % |
| Recuerde citar adecuadamente en su documentación cualquier información proveniente de fuentes bibliográficas. | |
| 3. Planeamiento | 10 % |
| 4. Uso de GitHub | 10 % |